

EKSAMENSFORSIDE

Hjemmeeksamen

Generell informasjon om eksamen:

Emnekode: OBJ2100**Emnenavn: Objektorientert programmering 2****Emneansvarlig: Tomas A. P. Olaj****Campus:**Ringerike (**Hønefoss**)**Fakultet: HH****Utleveringsdato og klokkeslett: 19.05.2025 klokken 12:00****Innleveringsdato og klokkeslett: 22.05.2025 innen klokken 12:00****Antall oppgaver: En hovedoppgave****Antall vedlegg: 0****Totalt antall sider (inkludert forside og vedlegg): 10**

Viktig informasjon ved alle typer hjemmeeksamen

HJELPEMIDLER OG SAMARBEID:

Tillatte hjelpemidler: Alle hjelpemidler er tillatt.

Eksamensform (kryss av): Individuell ☐ Gruppe ☒

På en hjemmeeksamen er det ikke lov til å samarbeide med andre grupper om hvordan dere skal skrive besvarelsen. Tekst, fremgangsmåte for oppgaveløsning, faglige refleksjoner osv. (besvarelsen deres) som leveres inn på eksamen skal være gruppens eget arbeid. Alle hjelpemidler er tillatt, og det kan brukes KI når dere svarer på oppgaven. Dersom dere bruker KI-verktøy til å generere tekst, bilder, kode, utregninger, løsningsforslag eller lignende må dere vise til det verktøyet dere har brukt. Dere må også redegjøre for, og noen ganger dokumentere, hvordan dere brukte KI-verktøyet. Her kan du lese mer om [USN's retningslinjer for bruk av KI](#). Vi gjør oppmerksom på at det kjøres plagiatskontroll på alle besvarelser, og at unaturlige likheter, ulovlig bruk av hjelpemidler, KI osv., eller at dere ikke har skrevet besvarelsen selv, fører til at USN oppretter sak om fusk. Fusk kan resultere i annullering av besvarelsen, tap av retten til å ta eksamen ved USN og andre universiteter og høyskoler og midlertidig utestenging fra USN i inntil to år. Beskrivelse av [individuell eksamen og ulovlig samarbeid finner dere på USN Intranett](#)

Eksamensoppgavetekst:

Oppgave

Et ledende konsultentselskap har fått i oppdrag å utvikle en applikasjon for **Nordisk Film Kino**, Norges største kinovirksomhet med over 3 millioner årlige besøkende og drift av 21 kinohus over hele landet. Applikasjonen skal støtte administrasjon, billettbestilling og salg på tvers av flere kinoer og kinosaler.

Applikasjonen skal støtte administrasjon, bestilling og salg av kinobilletter på tvers av flere kinoer og saler. Løsningen skal utvikles i **Java**, og kan benytte enten **MySQL** eller **PostgreSQL** som databasesystem, avhengig av studentenes valg.

Det er utarbeidet et databaseskript som er kompatibelt med begge løsningene. Pålogging til databasen skal kunne skje med brukernavn *Case* og passord *Esac*.

Det legges vekt på at applikasjonen er:

- Ryddig strukturert.
- Robust og feilfri i bruk.
- Dokumentert med god kodekvalitet og prinsipper for objektorientert utvikling.

Teknologikrav:

- **Programmeringsspråk:** Java
- **Database:** MySQL eller PostgreSQL
- **Backend:** Hovedfokus. Løsningen skal kunne kjøre og betjene alle funksjoner korrekt uten frontend.
- **Frontend:**

Dersom tid tillater det, kan studentene utvikle et grafisk brukergrensesnitt (GUI). Dette vil bli vurdert positivt, men er **ikke et krav** for å bestå.

Mulige GUI-teknologier inkluderer (basert på Liang-boken og annen praksis):

- **JavaFX** (moderne grafikkbibliotek i Java)
- **Swing** (eldre, men fortsatt mye brukt GUI-rammeverk)
- **AWT** (Abstract Window Toolkit som er enklere GUI)
- **Webbasert frontend** (f.eks. med HTML/CSS/JavaScript og REST-kommunikasjon)
- **Konsollbasert grensesnitt** (f.eks. menybasert terminalapplikasjon)

Applikasjonen skal fungere korrekt selv uten grafisk brukergrensesnitt.

Systemet skal ha fire hoveddeler:

1. For kinosentralens planlegger (Administrasjon)

- **Administrasjonsmodul:**
Planleggeren kan registrere nye filmer, opprette visninger, og endre eller slette eksisterende visninger så lenge det ikke er solgt billetter til dem.
- **Rapportmodul:**
Planleggeren kan generere statistikker over billettsalg, visningsbelegg og slettede bestillinger.

2. For kinobetjenten (Billettsalgsoperasjoner)

- **Betalingsmodul:**
Betjenten kan registrere at en forhåndsbestilt billett er betalt ved fysisk oppmøte.

- **Direktesalgmodul:**
Betjenten kan selge ledige billetter direkte til kunder i kinoen.
- **Avbestillingsmodul:**
Betjenten kan, 30 minutter før visningsstart, starte en prosess som sletter alle ubetalte bestillinger for en spesifikk visning.
Alle slettinger skal logges i filen `slettinger.dat`.

3. For kunden (Kinopublikum)

- **Bestillingsmodul:**
Kunden kan bestille billetter via selvbetjeningsløsninger i kinolokalet eller eksternt via andre terminaler.
Kunden får vist tilgjengelige visninger (der det er minst 30 minutter igjen til start), velger plasser, ser pris og antall, og mottar en unik billettcode ved bekreftelse.

4. Systemadministrasjon (Vedlikehold og sikkerhet)

- **Brukeradministrasjon:**
Mulighet for å administrere brukerkontoer for planleggere og kinobetjenter (opprettelse, sletting, endring av PIN-koder). Dokumenter gjerne hvordan.
- **Systemvedlikehold:**
Muligheter for å eksportere logger, ta sikkerhetskopier av databasen, rydde midlertidige data, og overvåke antall aktive bestillinger.
- Tilgang til denne delen er begrenset til brukere med utvidede administrative rettigheter. Her kan man som databaseadministrator legge til en kolonne manuelt i databasen for å skille ut systemadministratorer.
- **Dokumentasjon av sikkerhet og personvern**
Studentene skal i tillegg levere en kortfattet skriftlig vurdering (ca. ½–1 side) med tema:

Hvordan sikrer systemet personvern og datasikkerhet for ansatte og kunder?

Den skriftlige delen bør inneholde refleksjon rundt følgende punkter:

- Hvordan PIN-koder bør lagres (f.eks. med hashing – ikke i klartekst)
- Tilgangsstyring i systemet (hvem får gjøre hva?)
- Logging av sensitive operasjoner (f.eks. sletting av ubetalte bestillinger)
- Hvilke kundeopplysninger som kan anses som personopplysninger
- Hvordan systemet kan tilpasses krav fra GDPR (dataminimering, slette data ved forespørsel, etc.)

Når en kunde ønsker å bestille billetter, starter de bestillingsdelen uten å logge inn. De får da oversikt over kommende visninger hvor det er minst 30 minutter igjen til start, sammen med pris og antall ledige plasser. Visningene kan sorteres etter film eller tidspunkt. Når kunden har valgt en visning, vises de ledige enkeltplassene for denne visningen. Kunden kan velge – og eventuelt ombestemme – hvilke plasser de ønsker. Antall valgte plasser og totalbeløp oppdateres og vises fortløpende. Det er ikke et krav at systemet automatisk foreslår plasser.

For å finne visninger der det er minst 30 minutter til start, må programmet hente gjeldende klokkeslett og vise dette i brukergrensesnittet. Java tilbyr flere klasser for håndtering av tid, men det anbefales å bruke klassen `LocalDateTime` fra pakken `java.time`, da den er mer moderne og fleksibel enn den eldre `Date`. Et `LocalDateTime`-objekt kan opprettes med aktuell tid ved å bruke `LocalDateTime.now()`, og deretter konverteres til tekst med `toString()` eller ved hjelp av `DateTimeFormatter` for bedre kontroll på formatet. Dere må begrunne selv valgene. Her er et enkelt og pedagogisk eksempel på hvordan du kan hente aktuell tid i Java og formatere den, med moderne klasser fra `java.time`:

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Tidseksempel {
    public static void main(String[] args) {
        // Hent aktuell dato og tid
        LocalDateTime nå = LocalDateTime.now();

        // Definer ønsket format
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd.MM.yyyy HH:mm:ss");

        // Formater og skriv ut
        String formatertTid = nå.format(formatter);
        System.out.println("Aktuell tid: " + formatertTid);
    }
}
```

Når brukeren har valgt visning og seter, skal det vises et bekreftelsesskjema som inneholder alle relevante opplysninger, inkludert totalpris og generert billettcode.

På bekreftelsesskjemaet skal det også tydelig informeres om at billettene må hentes og betales senest 30 minutter før forestillingen starter, og at kunden må oppgi den viste billettcode ved henting. Det er ikke krav om å generere en fysisk utskrift av billetten.

Brukeren skal ha to valg:

- **Bekreft bestillingen** (bestillingen lagres og plassene reserveres).
- **Avbryt bestillingen** (de valgte plassene frigjøres).

Uansett valg skal en meldingsboks bekrefte handlingen, og systemet skal returnere brukeren til åpningsskjermen. Her kommer et forslag til en liten sekvensskisse over hvordan denne prosessen kan bygges opp i Java, f.eks. en liten flyt med metoder:

START

1. Bruker velger en visning → Controller kaller:
visTilgjengeligePlasser(visningId)
2. Bruker velger (og eventuelt ombestemmer) seter → Controller kaller:
oppdaterValgtePlasser(plassListe)
3. Systemet oppdaterer visning:
 - Antall valgte plasser
 - Totalpris
4. Når brukeren klikker "Bestill", åpnes bekreftelsesskjema → Controller kaller:
visBekreftelsesskjema(visning, valgtePlasser, totalPris)
5. Brukeren får to valg:
 - 5A. ****Bekreft bestilling****:
 - Generer billettcode
 - Lagre bestilling i databasen (tblBillett, tblPlassbillett)
 - Vis melding: "Bestilling bekreftet!"
 - Returner til hovedmenyen
 - 5B. ****Avbryt bestilling****:
 - Frigi reserverte plasser (ingen lagring i database)
 - Vis melding: "Bestilling avbrutt."
 - Returner til hovedmenyen

SLUTT

Når kundene ankommer kinoen for å hente og betale for sine forhåndsbestilte billetter, må de oppgi billettcode de fikk ved bestillingen. Kinobetjenten søker opp bestillingen basert på

billett-koden, mottar betaling, og oppdaterer deretter systemet slik at billettene registreres som betalt.

Operasjoner for kinobetjent og planlegger

Kinobetjentens oppgaver

- Kinobetjenten kan når som helst selge ledige billetter direkte til kunder. Slike billetter betales umiddelbart ved kjøp.
- Kinobetjenten kan, for en spesifikk visning, slette alle bestillinger som ikke er betalt.
 - Slettingen skal loggføres i en egen fil kalt `slettinger.dat`.
- For å få tilgang til disse funksjonene, må kinobetjenten logge inn i systemet med brukernavn og en firesifret PIN-kode.
 - Eksempelbruker: *tomas / 1234*.

Planleggerens oppgaver

- Kinosentralens planlegger må også logge inn med brukernavn og PIN-kode for å få tilgang til administrasjonsdelen av systemet.
 - Eksempelbruker: *samot / 4321*.
- Planleggeren kan:
 - Registrere, endre eller slette filmer og visninger, forutsatt at det ennå ikke er solgt billetter til visningen.
 - Generere statistikk:
 - For en bestemt film:
Vise utviklingen over tid, inkludert antall besøkende og belegg i prosent av salens kapasitet per visning, samt antall bestillinger som ble slettet fordi de ikke ble betalt/hentet i tide.
 - For en bestemt kinosal:
Vise hvor mange prosent av plassene som i gjennomsnitt ble solgt for visninger som har vært avholdt.

Om databasen

Det forutsettes at programvaren for ønsket database er installert. MySQL anbefalt versjon: 8.x, eller PostgreSQL anbefalt versjon er 14 eller nyere. Databasen må manuelt opprettes med navn **kino**.

- Et skript som genererer databasen med nødvendige tabeller og noen eksemplariske data, er vedlagt på forhånd i **Canvas (OBJ2100: våren 2025)**. Filene i Canvas vil typisk hete:
 - `kino_postgres_v?_final.sql`
 - `kino_mysql_v?_final.sql`
- Det må kanskje gjøres noen tilpasninger for at brukeren Case skal kunne koble til databasen **kino**, og få full tilgang til tabeller og sekvenser i skjemaet. Dokumenter eventuelle tilpasninger. Etter det skal databasen anses som ferdig og låst etter opprettelse. Typisk er det interne database rettigheter, ekstern tilgang, JDBC-driver i Java-applikasjonen, og kode for tilkobling.
- Studenter kan legge inn flere testdata dersom ønskelig, men selve Java-applikasjonen skal ikke:
 - **endre databasens struktur** (f.eks. legge til/fjerne kolonner eller tabeller).
 - **lagre SQL-spørringer direkte i databasen** for bruk i applikasjonen. Alle databasetilganger skal håndteres via Java-kode.
- Riktig JDBC-driver må være lagt til i prosjektet, og JDBC-URL i Java må peke til riktig database.

```

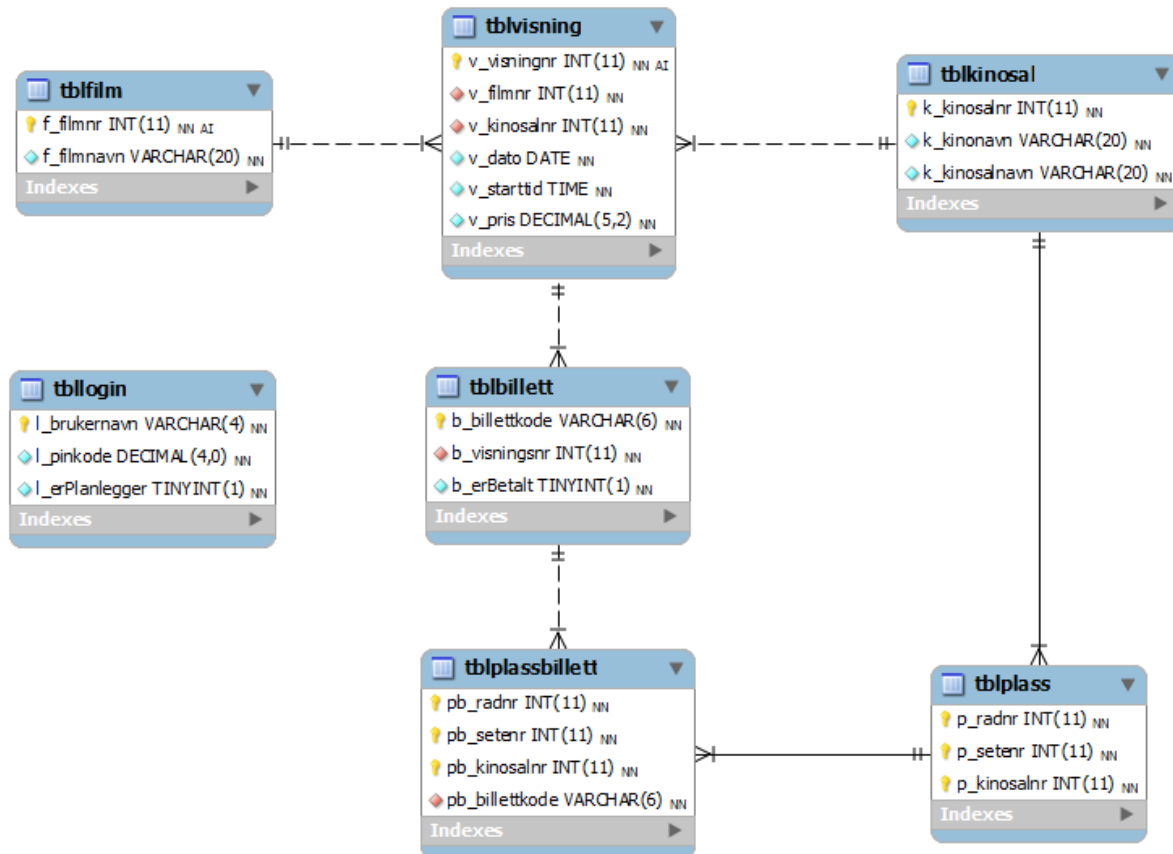
tola@E-5CD2032B50:~$ psql -U Case -d kino -h localhost -p 5432
Password for user Case:
psql (16.8 (Ubuntu 16.8-0ubuntu0.24.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

kino=> \dt kino.*
               List of relations
 Schema |      Name      | Type  | Owner
-----+-----+-----+-----
 kino  | tblbillett     | table | postgres
 kino  | tblfilm        | table | postgres
 kino  | tblkinosal     | table | postgres
 kino  | tbllogin       | table | postgres
 kino  | tblplass       | table | postgres
 kino  | tblplassbillett | table | postgres
 kino  | tblvisning     | table | postgres
(7 rows)

kino=>

```

Datamodellen ser slik ut ¹:



¹ MySQLs `TINYINT(1)` tilsvarer Java's boolean.

Datamodellen kan forbedres selv om den er brukbar som den er. Det er lov å gjøre forbedringer og forslag kan være:

1. Datatyper og sikkerhet

- `l_pinkode` er av typen `DECIMAL(4, 0)`
 - ✂ Bytt til **VARCHAR(4)** eller `CHAR(4)` for fleksibilitet, og hash PIN-en i Java.
 - `DECIMAL` er tall og kan miste ledende nuller (f.eks. `0123` → `123`).

2. Brukerroller mer fleksibelt

- `l_erPlanlegger` som `TINYINT(1)` er greit, men begrenset.
 - ✂ Bytt til role **VARCHAR(20)**, med verdier som: "admin", "planlegger", "betjent".
 - Dette gir rom for fremtidige roller (f.eks. "support", "superbruker").

3. Manglende primærnøkkel i `tblplassbillett`

- Tabellen `tblplassbillett` mangler definert primærnøkkel. ✂ Anbefalt sammensatt PK: (`pb_billettkode`, `pb_radnr`, `pb_setenr`, `pb_kinosalnr`)
 - Dette sikrer unik kobling mellom billett og sete i riktig sal.

4. Indekser

- Sørg for at kolonner som brukes i `WHERE`-søk er indeksert, f.eks.:
 - `b_billettkode` i `tblbillett`
 - `l_brukernavn` i `tblLogin`
 - `v_filmnr`, `v_kinosalnr` i `tblvisning`

5. Datakonsistens og validering

- Det finnes ingen felter for f.eks. dato opprettet, tidspunkt for betaling eller sletting.
 - ✂ Det er ikke nødvendig i dette oppgavesettet, men kunne vært nyttig for logging og revisjon.

Merknader om databaseinnhold og håndtering i applikasjonen

Opplysningene i tabellene `tblLogin`, `tblKinosal` og `tblPlass` anses som relativt statiske, og skal derfor legges direkte inn i databasen manuelt (f.eks. via skript eller databaseverktøy). Disse dataene skal **ikke** redigeres gjennom Java-applikasjonen.

De øvrige tabellene (`tblFilm`, `tblVisning`, `tblBillett`, `tblPlassBillett`) skal **kun håndteres gjennom applikasjonen**, som gir funksjonalitet for å opprette, lese, endre og slette data der det er relevant.

Plassidentitet og struktur

Hver enkelt plass i en kinosal identifiseres ved en kombinasjon av:

- **Radnummer** (`p_radnr`) – starter fra 1
- **Setenummer** (`p_setenr`) – starter fra 1 for hver rad
- **Kinosalnummer** (`p_kinosalnr`) – peker til riktig sal

Billett-koder

Attributtet `b_billettkode` i tabellen `tblBillett` er en systemgenerert, unik verdi bestående av **fire store bokstaver etterfulgt av to siffer** (f.eks. ABCD12).

Denne verdien er satt som **primærnøkkel** i databasen, noe som betyr at **programmet må sikre at verdien er unik** før den lagres.

Billett-koden kunne vært generert automatisk i databasen (for eksempel via `UUID()` eller `AUTO_INCREMENT`), men dette er ikke implementert. Det er derfor **studentenes ansvar å implementere logikk i Java** for å generere en ny kode, og sikre at den ikke kolliderer med eksisterende verdier. Dersom en kollisjon oppstår (dvs. billett-koden allerede finnes), må programmet generere en ny kode automatisk – **uten å gi brukeren feilmelding**.

Om `tblPlassbillett`

Tabellen `tblPlassbillett` inneholder en oversikt over hvilke konkrete plasser som er reservert eller betalt for en gitt visning.

Den inkluderer kun de plassene som er **aktivt bestilt** (enten betalt eller ikke), og er avgjørende for å vite hvilke seter som er ledige ved visningstidspunktet.

Krav til design av systemet

Bruker-grensesnitt (valgfritt – gir tilleggspoeng)

Utvikling av grafisk bruker-grensesnitt (GUI) er **valgfritt**, men vil telle positivt ved vurdering. Dersom dere velger å implementere GUI, gjelder følgende felles retningslinjer:

- Programmet skal være tydelig **oppdelt i lag**:
 - **Grensesnittlag (View)**
 - **Kontrollag (Controller)**
 - **Domenelag (Model)**
- Det skal finnes et **hovedvindu** hvor brukeren kan velge hvilken del av systemet som skal brukes (kunde, kinobetjent, planlegger).
- Hver del av grensesnittet skal vises i en **egen scene** eller visningsflate.
- **Rapportdelen** skal presentere statistikk over billettsalg i **tabellform**.
- Ved søk etter forestillinger skal **aktuell tid vises** i grensesnittet, gjerne formatert.

Datainnhenting og -oppdatering

Uavhengig av om GUI implementeres eller ikke, gjelder følgende krav til databehandling:

- Ved **programstart** skal alle nødvendige data leses inn fra databasen og lagres i egnede datastrukturer i minnet.
 - Det anbefales å bruke for eksempel `ArrayList`-objekter for å representere lister over filmer, visninger og billetter.
- Endringer i databasen (f.eks. nye billetter, slettede bestillinger og oppdaterte visninger) skal **ikke** lagres kontinuerlig, men først **skrive tilbake til databasen når programmet avsluttes**.

Bruk av Java-rammeverk og eksterne biblioteker

Studentene står fritt til å benytte relevante **Java-rammeverk og biblioteker** for å løse oppgaven mer effektivt. Det er for eksempel tillatt å bruke:

- **Spring Boot** for REST-tjenester eller datatilgang
- **JavaFX, Swing** eller andre GUI-rammeverk
- **Eksterne API-er** eller bibliotek som f.eks. Apache Commons, Gson, Jackson m.m.
- **Maven** eller **Gradle** for å håndtere avhengigheter

Bruk av slike teknologier **er ikke påkrevd**, men vil kunne bidra positivt til vurderingen dersom det er hensiktsmessig brukt, godt dokumentert og forståelig.

Merk: Det forutsettes at all brukt kode kan bygges og kjøres uten nettilgang, og at nødvendige avhengigheter inkluderes i prosjektet ved innlevering.

Formelle krav

Oppgaven utleveres **mandag 19. mai kl. 12.00 i WiseFlow**. Alle hjelpemidler er tillatt.

Prosjektet kan løses individuelt eller i grupper på inntil seks studenter. Det er tillatt å levere som én student, men dette anbefales **ikke**, da prosjektets omfang og kompleksitet tilsier at det kreves god arbeidsdeling og samarbeid for å levere et fullverdig produkt. Erfaring viser at en gruppe på minst tre deltakere gir best forutsetninger for å lykkes.

Studentene setter selv sammen gruppene. Ved innlevering leveres prosjektet av én deltaker som deretter **inviterer resten av gruppen** i innsendingstjenesten (f.eks. WiseFlow).

Merk: Jo flere studenter som deltar i en gruppe, desto høyere stilles kravene til prosjektets kvalitet, funksjonalitet og dokumentasjon. Dette vil bli tatt hensyn til i vurderingen.

Krav til innlevering

Besvarelsen leveres som zip-fil av prosjektet og annen dokumentasjon (pdf) i WiseFlow **senest torsdag 12. juni kl. 12.00**. Fristen er absolutt. Det betyr at dere leverer det dere har klart.

Følgende skal leveres:

1. Fullt kjørbart system

- Prosjektet skal være komplett og kunne kjøres uten ytterligere endringer.
- Endringer i database-scriptet må dokumenteres eller hele scriptet bør legges ved, pga. sensor tar utgangspunkt i sin egen kopi av databasen, så **systemet må fungere uavhengig av lokale testdata**.
- Det er **ikke nødvendig å kompilere** programmet på forhånd – sensor gjør dette selv.
- **Pass på at alle nødvendige biblioteker er inkludert**, som:
 - JDBC-driver for MySQL eller PostgreSQL
 - JavaFX-biblioteker (dersom GUI er brukt)
- **Kontroller at alle filer er med i ZIP-arkivet** før innlevering.

2. Dokumentasjon

a) Kildekode

- Lever fullstendig og kommentert **Java-kildekode** (. java-filer).
- Benytt **JavaDoc-kommentarer** for klasser og metoder.
- Det skal fremgå av kommentaren:
 - Hvem som har utviklet koden
 - Hvem som har testet og eventuelt godkjent den

! Ikke lim inn kildekoden i egne dokumenter – lever den i original form som . java-filer.

b) Prosjektrapport (enkelt teknisk dokument i PDF)

Rapporten skal inneholde:

- Gruppens navn (valgfritt), fullt navn/eller studentnummer og kandidatnummer for alle deltakere.
- Hvordan prosjektarbeidet ble organisert og fordelt.
- **En tydelig oversikt over hva hver deltaker konkret har gjort**
(Dette er viktig for å vise individuell innsats og rolleforståelse i gruppeprosjektet, selv om alle i gruppen får samme karakter. Det hjelper også sensor å vurdere arbeidsdelingen og prosjektets gjennomføring.)
- Beskrivelse av tekniske eller organisatoriske utfordringer gruppen har møtt og hvordan de ble løst.
- Hjelp og kilder som gruppen har benyttet underveis.

Det er også viktig at rapporten beskriver hvordan dere har analysert oppgaven og brutt caset ned i deler – f.eks. hvilke klasser, moduler eller komponenter som inngår i systemet (som "Film", "Visning", "Billett", "Brukerhåndtering", "Rapportering" osv.).

Det bør gå frem **hvordan dere har strukturert systemet objektorientert**, og hvilke valg dere har tatt for å gjøre det forståelig og utvidbart.

Leveringsformat

- Hele prosjektet og dokumentasjonen skal pakkes i **standard ZIP-format**. Dobbeltsjekk at ZIP-filen ikke er korrupt.
- Det er mulig at Wizeflow med FLOWassign er satt opp med innlevering av pdf, og vedlegg (hvor ZIP-filen legges ved).
- Kun én ZIP-fil skal leveres i WizeFlow – én gang – av én av gruppemedlemmene.
- Denne studenten **inviterer de øvrige gruppemedlemmene** til innsendingen etter opplasting.

Pass på at det er **hele** prosjektet dere zipper. Dere kan tolke denne oppgaven og ta forutsetninger hvis dere synes det er behov for det. **Forutsetninger** skal komme frem av prosjektrapporten. Er dere usikre, send spørsmål via Canvas. Velg selv passende brukergrensesnitt, feilmeldinger og annet. Hvis du får ekstra tid, bør du heller forsøke å bedre kvaliteten enn å utvide funksjonaliteten. Ikke et krav, men sensor vil ta utgangspunkt i IntelliJ plattformen.

Vurderingskriterier

Prosjektet vurderes etter karakterskala A–F i henhold til USNs prosentskala:

- A: 89–100 %, B: 77–88 %, C: 65–76 %, D: 53–64 %, E: 40–52 %, F: 0–39 %

Følgende kriterier inngår i vurderingen og vektes som angitt:

Vurderingsområde	Vekt
1. Klasseutforming og datamodellkobling	25 %
2. Databaseinteraksjon (CRUD, spørringer, håndtering)	15 %
3. Funksjonalitet i billettsystemet	15 %
4. Håndtering av samtidighet	10 %
5. Bruk av designmønstre	10 %
6. Oppfyllelse av systemkrav og brukergrensesnitt	10 %
7. Dokumentasjon og prosjektrapport	15 %

Lykke til med eksamensprosjektet!